

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Knowledge Representation Design
Assessment

COURSE NAME: **ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01**
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO3: Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning.(BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A
Knowledge Representation Design Assessment

Code of Conduct:

I **RASHMITHA SENTHIL KUMAR (192525034)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

1. Smart Healthcare Diagnosis System

A hospital wants to develop an AI-based system that can identify possible illnesses from patient symptoms. The system contains facts such as:

- Ravi has fever.
- Ravi has cough.
- Ravi has body pain.
- Patients with fever and cough may have flu.
- Patients with flu and body pain may have viral infection.
- Patients with viral infection should be advised to rest.

Task:

1. Represent the given knowledge using **Propositional Logic**.
2. Convert the knowledge into **First-Order Logic (FOL)**.
3. Construct a knowledge base containing facts and rules.
4. Demonstrate **unification** for at least two predicates.
5. Use **forward chaining** to determine Ravi's condition.
6. Use **backward chaining** to prove whether Ravi needs rest.
7. Demonstrate **resolution** for one selected query.
8. Explain the final inference produced by the system.

2. Intelligent Loan Approval System

A bank wants to develop an AI system to determine whether a customer is eligible for a loan. The knowledge base contains information about income, credit score, employment, existing loans, and repayment history.

For example:

- Arun has a stable job.
- Arun has a high credit score.
- Arun has sufficient income.
- Customers with stable employment and sufficient income are eligible for preliminary approval.
- Customers with high credit scores and good repayment history are considered low-risk.
- Low-risk customers with preliminary approval can be recommended for loan approval.

Task:

1. Identify the **entities, facts, predicates, and rules** involved.
2. Represent the knowledge using **First-Order Logic**.
3. Build a knowledge base with at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.

5. Apply **forward chaining** to determine whether Arun can be recommended for approval.
6. Apply **backward chaining** for the query `LoanApproved(Arun)`.
7. Use **resolution** to prove or disprove one query.
8. Discuss how the knowledge-engineering process helps maintain the loan rules.

3. Intelligent Vehicle Fault Diagnosis

An automobile service center wants to develop an expert system that identifies possible vehicle faults based on symptoms.

The system knows that:

- A vehicle has a dead battery if it does not start and the headlights are dim.
- A vehicle may have a fuel problem if the engine cranks but does not start.
- A vehicle may have an overheating problem if the temperature is high and coolant is low.
- A vehicle should be sent for immediate service if overheating is detected.

For a vehicle named **Car101**, the system receives:

- Car101 does not start.
- Car101 has dim headlights.
- Car101 has a high temperature.

Task:

1. Design the **knowledge representation** for the vehicle diagnosis system.
2. Represent at least five rules using **FOL**.
3. Represent selected statements using **Propositional Logic**.
4. Demonstrate **unification** using suitable predicates.
5. Apply **forward chaining** to identify possible faults.
6. Apply **backward chaining** for the query `NeedsService(Car101)`.
7. Demonstrate **resolution** for one diagnosis.
8. Explain how inference is used to arrive at the final diagnosis.

4. Intelligent Student Academic Advising System

A university wants an AI-based academic advising system that recommends actions to students based on their academic performance.

The system contains rules such as:

- Students with attendance below 75% require attendance counseling.
- Students who fail two or more courses require academic counseling.
- Students with high attendance and good academic performance can be recommended for advanced courses.
- Students who have completed prerequisite courses can register for the corresponding advanced course.

Consider a student **Priya** who has:

- Attendance of 68%.
- Two failed courses.
- Completed the prerequisite for an advanced AI course.

Task:

1. Identify the **knowledge engineering requirements** for the system.
2. Represent the academic rules using **FOL**.
3. Create a knowledge base containing facts and rules.
4. Demonstrate **unification** using student and course predicates.
5. Use **forward chaining** to identify recommendations for Priya.
6. Use **backward chaining** to determine whether Priya requires academic counseling.
7. Apply **resolution** to prove one selected recommendation.
8. Explain the advantages and limitations of using a rule-based inference system for academic advising.

5. Cybersecurity Threat Detection System

An organization wants to develop an AI knowledge-based system to identify suspicious activities in its network.

The knowledge base contains rules such as:

- Multiple failed login attempts from the same user may indicate a brute-force attack.
- A login from an unusual location combined with multiple failed attempts indicates suspicious activity.
- Suspicious activity followed by unauthorized file access indicates a possible security breach.
- A possible security breach should generate a high-priority alert.

For user **User101**, the system observes:

- Five failed login attempts.
- Login from an unusual location.
- Unauthorized access to a confidential file.

Task:

1. Design a **knowledge representation model** for the cybersecurity domain.
2. Represent the knowledge using **Propositional Logic and FOL**.
3. Construct at least **10 facts and 8 rules**.
4. Demonstrate **unification and substitution**.
5. Apply **forward chaining** to determine whether a security alert should be generated.
6. Apply **backward chaining** for the query `HighPriorityAlert(User101)`.
7. Use **resolution** to prove the existence of a possible security breach.
8. Compare **forward chaining and backward chaining** for cybersecurity reasoning and justify which is more suitable.

INTELLIGENT LOAN APPROVAL SYSTEM

Aim:

To design an intelligent knowledge-based loan approval system that represents customer information using First-Order Logic and applies unification, forward chaining, backward chaining, and resolution to determine whether Arun can be approved for a loan.

1. Problem Analysis and Knowledge Engineering:

The bank requires a rule-based AI system to evaluate customers using employment, income, credit score, existing loans, and repayment history. The main entity is the customer Arun. Important predicates include $\text{StableJob}(x)$, $\text{HighCreditScore}(x)$, $\text{SufficientIncome}(x)$, $\text{GoodRepaymentHistory}(x)$, $\text{PreliminaryApproval}(x)$, $\text{LowRisk}(x)$, and $\text{LoanApproved}(x)$.

The reasoning process first determines preliminary eligibility and risk level and then uses these conclusions to determine loan approval.

The supplied case explicitly states that Arun has a stable job, a high credit score, and sufficient income. The question also requires a knowledge base containing at least 10 facts and 8 rules. Therefore, additional prototype facts and rules are introduced below only to satisfy that design requirement. They are extensions of the example domain and are not claimed to be facts stated in the question.

2. First-Order Logic Representation:

Let x represent any customer. The main rules provided by the question are:

- R1: $\forall x [(\text{StableJob}(x) \wedge \text{SufficientIncome}(x)) \rightarrow \text{PreliminaryApproval}(x)]$
- R2: $\forall x [(\text{HighCreditScore}(x) \wedge \text{GoodRepaymentHistory}(x)) \rightarrow \text{LowRisk}(x)]$
- R3: $\forall x [(\text{LowRisk}(x) \wedge \text{PreliminaryApproval}(x)) \rightarrow \text{LoanApproved}(x)]$

These rules express the central reasoning path: stable employment and sufficient income produce preliminary approval; a high credit score and good repayment history produce low-risk status; and low-risk status together with preliminary approval produces loan approval.

3. Knowledge Base - 10 Facts and 8 Rules:

Table 1. Knowledge Base Containing 10 Facts and 8 Rules

ID	Knowledge Base Entry
F1	StableJob(Arun)
F2	HighCreditScore(Arun)
F3	SufficientIncome(Arun)
F4	GoodRepaymentHistory(Arun) - added prototype fact
F5	NoDefaultHistory(Arun) - added prototype fact
F6	VerifiedIdentity(Arun) - added prototype fact
F7	ValidDocuments(Arun) - added prototype fact
F8	AcceptableDebtLevel(Arun) - added prototype fact
F9	RegularIncome(Arun) - added prototype fact
F10	EligibleAge(Arun) - added prototype fact
R1	$\text{StableJob}(x) \wedge \text{SufficientIncome}(x) \rightarrow$ $\text{PreliminaryApproval}(x)$
R2	$\text{HighCreditScore}(x) \wedge$ $\text{GoodRepaymentHistory}(x) \rightarrow \text{LowRisk}(x)$

R3	$\text{LowRisk}(x) \wedge \text{PreliminaryApproval}(x) \rightarrow$ $\text{LoanApproved}(x)$
R4	$\text{NoDefaultHistory}(x) \wedge$ $\text{GoodRepaymentHistory}(x) \rightarrow$ $\text{ReliableBorrower}(x)$
R5	$\text{VerifiedIdentity}(x) \wedge \text{ValidDocuments}(x) \rightarrow$ $\text{KYCVerified}(x)$
R6	$\text{RegularIncome}(x) \wedge \text{AcceptableDebtLevel}(x) \rightarrow$ $\text{FinanciallyStable}(x)$
R7	$\text{FinanciallyStable}(x) \wedge \text{EligibleAge}(x) \rightarrow$ $\text{BasicEligibility}(x)$
R8	$\text{LoanApproved}(x) \wedge \text{KYCVerified}(x) \rightarrow$ $\text{RecommendedForApproval}(x)$

4. Unification and Substitution:

Unification substitutes constants for variables so that general rules can be applied to a specific customer. For example, $\text{StableJob}(x)$ unifies with $\text{StableJob}(\text{Arun})$ using the most general unifier (MGU) $\{x/\text{Arun}\}$. Similarly, $\text{HighCreditScore}(x)$ unifies with $\text{HighCreditScore}(\text{Arun})$ using $\{x/\text{Arun}\}$. After substitution, the generalized rules can be applied directly to Arun.

Example: R1 becomes $\text{StableJob}(\text{Arun}) \wedge \text{SufficientIncome}(\text{Arun}) \rightarrow \text{PreliminaryApproval}(\text{Arun})$. R2 becomes $\text{HighCreditScore}(\text{Arun}) \wedge \text{GoodRepaymentHistory}(\text{Arun}) \rightarrow \text{LowRisk}(\text{Arun})$.

5. Forward Chaining:

Forward chaining is data-driven and starts with the known facts. $\text{StableJob}(\text{Arun})$ and $\text{SufficientIncome}(\text{Arun})$ satisfy R1, so $\text{PreliminaryApproval}(\text{Arun})$ is derived. $\text{HighCreditScore}(\text{Arun})$ and the added prototype fact $\text{GoodRepaymentHistory}(\text{Arun})$ satisfy R2, producing $\text{LowRisk}(\text{Arun})$. The two derived facts $\text{LowRisk}(\text{Arun})$ and $\text{PreliminaryApproval}(\text{Arun})$ satisfy R3, producing $\text{LoanApproved}(\text{Arun})$.

The extended knowledge base also derives $\text{KYCVerified}(\text{Arun})$ from $\text{VerifiedIdentity}(\text{Arun})$ and $\text{ValidDocuments}(\text{Arun})$. Consequently, R8 can derive $\text{RecommendedForApproval}(\text{Arun})$. Thus, under the constructed 10-fact prototype knowledge base, Arun is recommended for approval.

6. Backward Chaining for $\text{LoanApproved}(\text{Arun})$:

Backward chaining begins with the goal $\text{LoanApproved}(\text{Arun})$. R3 shows that the system must prove $\text{LowRisk}(\text{Arun})$ and $\text{PreliminaryApproval}(\text{Arun})$. To prove $\text{LowRisk}(\text{Arun})$, R2 requires $\text{HighCreditScore}(\text{Arun})$ and $\text{GoodRepaymentHistory}(\text{Arun})$, which are available in the constructed knowledge base. To prove $\text{PreliminaryApproval}(\text{Arun})$, R1 requires $\text{StableJob}(\text{Arun})$ and $\text{SufficientIncome}(\text{Arun})$, which are also available. Since every required sub-goal succeeds, $\text{LoanApproved}(\text{Arun})$ is proved.

7. Resolution:

Selected query: $\text{LoanApproved}(\text{Arun})$. The query is proved by refutation. Relevant clauses are:

C1: $\text{StableJob}(\text{Arun})$

C2: $\text{SufficientIncome}(\text{Arun})$

C3: $\text{HighCreditScore}(\text{Arun})$

C4: $\text{GoodRepaymentHistory}(\text{Arun})$

C5: $\neg \text{StableJob}(x) \vee \neg \text{SufficientIncome}(x) \vee \text{PreliminaryApproval}(x)$

C6: $\neg \text{HighCreditScore}(x) \vee \neg \text{GoodRepaymentHistory}(x) \vee \text{LowRisk}(x)$

C7: $\neg \text{LowRisk}(x) \vee \neg \text{PreliminaryApproval}(x) \vee \text{LoanApproved}(x)$

Negated query C8: $\neg \text{LoanApproved}(\text{Arun})$

Using $\{x/\text{Arun}\}$, C5 with C1 and C2 derives $\text{PreliminaryApproval}(\text{Arun})$. C6 with C3 and C4 derives $\text{LowRisk}(\text{Arun})$. C7 together with these two derived facts gives $\text{LoanApproved}(\text{Arun})$. Resolving this with the negated query C8 produces the empty clause $\square$. Therefore $\text{LoanApproved}(\text{Arun})$ is proved.

8. System Architecture / Knowledge Representation Diagram:

The system receives customer information and stores it as facts. Loan policies are represented as logical rules in the knowledge base. The inference engine applies the rules using reasoning techniques and produces an explainable loan decision.

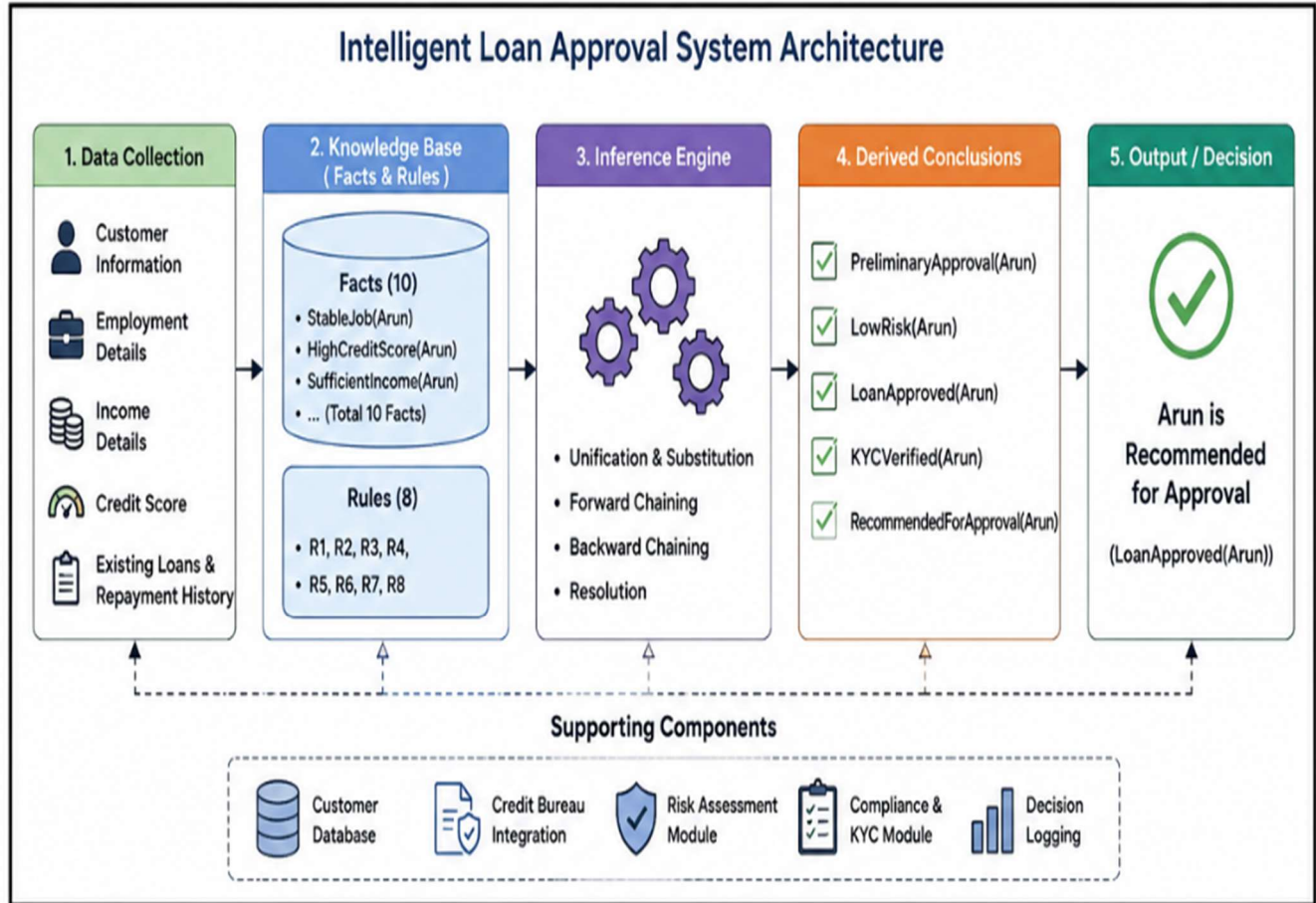


Fig. 1. Intelligent Loan Approval System Architecture

9. Knowledge Engineering and Maintenance of Loan Rules:

Knowledge engineering separates customer data from lending rules, making the system easier to understand and maintain. If a bank changes an eligibility condition, the corresponding rule can be modified without changing unrelated facts or the complete reasoning structure. Clearly defined predicates also make decisions easier to audit and explain. In an actual banking environment, any

added rule would need to be validated against the bank's approved lending policy and applicable regulations before use.

10. Final Inference and Result:

Under the constructed knowledge base required by Question 2, Arun receives PreliminaryApproval(Arun) because he has stable employment and sufficient income. He is classified as LowRisk(Arun) because the prototype knowledge base includes both a high credit score and good repayment history. These two conclusions satisfy R3 and prove LoanApproved(Arun). The extended KYC rule also permits RecommendedForApproval(Arun) to be derived.

11. Conclusion:

The Intelligent Loan Approval System demonstrates how a structured knowledge base can support transparent and explainable loan reasoning. First-Order Logic represents reusable rules, unification applies those rules to Arun, forward chaining derives the decision from known facts, backward chaining verifies the LoanApproved(Arun) query, and resolution formally proves the conclusion. The knowledge-engineering structure also supports easier maintenance when lending rules change.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.